
Plan DevOps y Guía de Comunicación

Proyecto Móvil: RentaFácil (Plataforma PropTech en Flutter)

Docente: Dora Lilia López Angeles
Proyecto: RentaFácil App
Versión del Plan: 1.1.0

Líder de Proyecto / Dev: [Tu Nombre /
Matrícula]
Fecha de Entrega: Septiembre 2026
Estado: Documento Integral Aprobado

Índice

1. Resumen del Caso y Objetivos	3
1.1. Descripción del Caso de Uso	3
1.2. Métricas de Éxito y KPIs	3
2. Alcance Inicial (Sprint 0) y Backlog Priorizado	3
2.1. Historias de Usuario (INVEST) y Criterios de Aceptación	3
2.2. Estructura del Tablero de Trabajo (Board)	3
3. Arquitectura General y Dependencias	4
3.1. Visión de Arquitectura de Alto Nivel	4
3.2. Matriz de Dependencias Técnicas	4
4. Ciclo de Vida DevOps Aplicado	4
5. Estrategia de Versionamiento y Colaboración	4
5.1. Modelo de Ramificación (GitFlow Ligero)	4
5.2. Convención de Commits (Conventional Commits)	5
5.3. Versionamiento Semántico (SemVer)	5
6. Integración y Entrega Continua (CI/CD)	5
6.1. Pipeline Automatizado de CI (GitHub Actions)	5
6.2. Criterios de Merge	6
7. Estrategia de Pruebas	6
8. Estrategia de Despliegue	6

9. Monitoreo, Métricas y Operabilidad	6
10. Matriz de Riesgos y Mitigaciones	6
11. Runbook Corto de Respuesta a Fallas Comunes	7
12. Investigación, Selección y Normas de Comunicación	7
12.1. Matriz Comparativa de Herramientas de Comunicación	7
12.2. Dictamen de Selección (≤ 300 palabras)	7
12.3. Configuración de Canales y Playbook de Uso	7

1 Resumen del Caso y Objetivos

1.1 Descripción del Caso de Uso

El mercado de arrendamiento residencial y comercial padece altas fricciones operativas debido a procesos manuales, baja verificación de perfiles e informales tiempos de respuesta que promedian más de 72 horas. **RentaFácil** es una plataforma móvil y web responsiva desarrollada en **Flutter** conectada a una arquitectura Backend REST API con **PostgreSQL**.

La solución está enfocada en dos segmentos clave: *inquilinos* (profesionales y estudiantes que demandan inmediatez y seguridad) y *propietarios* (que requieren reducir la vacancia de sus propiedades y validar el perfil crediticio de sus inquilinos). La propuesta de valor elimina intermediarios tradicionales, integra validación de identidad (KYC) y gestiona contratos y pagos digitales de forma automatizada.

1.2 Métricas de Éxito y KPIs

- **Lead Time de Reserva (Negocio):** Reducción del tiempo transcurrido desde la primera solicitud de contacto hasta la firma digital del contrato, pasando de 72 horas a menos de 6 horas.
- **Tasa de Usuarios Libres de Fallos / Crash-Free Users (Calidad):** Mantener un umbral continuo $\geq 99,2\%$ de sesiones activas sin cierres inesperados en entornos de producción.
- **Frecuencia de Despliegue y Tiempo de Recuperación / MTTR (DevOps):** Lograr pases a producción en menos de 15 minutos mediante pipelines automatizados y un MTTR < 30 minutos ante incidentes críticos.

2 Alcance Inicial (Sprint 0) y Backlog Priorizado

2.1 Historias de Usuario (INVEST) y Criterios de Aceptación

ID	Historia de Usuario	Criterios de Aceptación (Gherkin)	MoSCoW	WSJF
US-01	Autenticación e Inicio de Sesión: Como usuario quiero ingresar con email o Google.	Given usuario no autenticado, When ingresa credenciales válidas, Then se genera JWT y redirige al Dashboard.	Must Have	9.5
US-02	Publicación de Inmueble: Como propietario quiero cargar fotos y datos de mi propiedad.	Given propietario verificado, When sube ≥ 3 fotos y datos, Then el inmueble cambia a estado "Publicado".	Must Have	9.0
US-03	Búsqueda y Filtros: Como inquilino quiero filtrar inmuebles por precio y zona.	Given buscador activo, When se aplica filtro de precio, Then la lista responde en < 1 segundo.	Must Have	8.5
US-04	Detalle de Propiedad: Como inquilino quiero ver la galería y reglas del inmueble.	Given selección de tarjeta, When abre la vista, Then muestra carrusel, mapa y botón de solicitud.	Must Have	8.0
US-05	Solicitud de Renta: Como inquilino quiero contactar al dueño mediante la app.	Given detalle de inmueble, When presiona "Solicitar", Then notifica al dueño y abre chat.	Should Have	7.0
US-06	Verificación KYC: Como usuario quiero validar mi ID oficial para tener la insignia.	Given vista de perfil, When sube foto de ID, Then el API valida y asigna insignia "Verificado".	Should Have	6.5

Cuadro 1: Backlog de Historias de Usuario para el Sprint 0.

2.2 Estructura del Tablero de Trabajo (Board)

El flujo Kanban configurado comprende 5 columnas: **Backlog** (US-05 a US-08), **To-Do** (US-01 a US-04 reservadas para Sprint 0), **In-Progress** (Setup de arquitectura), **Review** (PRs abiertas esperando checks) y **Done** (Código integrado en 'develop').

3 Arquitectura General y Dependencias

3.1 Visión de Arquitectura de Alto Nivel

- **Cliente Móvil y Web:** Aplicación compilada en Flutter SDK (Dart) implementando Clean Architecture (Capa de Presentación, Dominio y Datos con manejo de estado BLoC/Riverpod).
- **Capa de API REST Backend:** Servicio desacoplado desplegado en contenedores Docker sobre arquitectura AWS/GCP con balanceador de carga.
- **Capa de Persistencia:** Base de datos PostgreSQL para almacenamiento relacional del catálogo, usuarios y contratos.

3.2 Matriz de Dependencias Técnicas

Categoría	Tecnología / Servicio	Propósito en el Proyecto
SDK / Lenguaje	Flutter 3.x / Dart 3.x	Desarrollo multiplataforma UI.
Base de Datos	PostgreSQL 15	Almacenamiento relacional estricto.
Autenticación	Firebase Auth / JWT	Gestión segura de sesiones de usuario.
Servicios KYC	Truora / Jumio API	Validación de identificaciones oficiales.
Firma Digital	Mifiel / DocuSign API	Contratos de arrendamiento legalmente válidos.
Monitoreo	Sentry / Firebase Crashlytics	Rastreo de excepciones y analítica de rendimiento.

Cuadro 2: Matriz de componentes y dependencias del sistema.

4 Ciclo de Vida DevOps Aplicado

Fase	Objetivo	Herramientas	Artefactos / Salidas	Criterio de Hecho (DoD)
1. Plan	Gestionar Backlog e iteraciones.	GitHub Projects	Historias INVEST	Criterios Gherkin aprobados.
2. Code	Desarrollo modular y control de cambios.	VS Code / Git	Commits / PRs	Aprobación de 1 revisor par.
3. Build	Compilación y análisis estático.	GitHub Actions	Executable intermedio	Linter finalizado sin warnings.
4. Test	Ejecución de pruebas automatizadas.	Flutter Test / Mocktail	Reporte Cobertura	Cobertura unitaria $\geq 80\%$.
5. Package	Empaquetado y firmado de binarios.	Fastlane / Gradle	APK / AAB / IPA	Binario firmado generado.
6. Release	Etiquetado y notas de versión.	GitHub Releases	Git Tag / Changelog	Etiqueta SemVer ('vX.Y.Z').
7. Deploy	Distribución a tiendas y betas.	Firebase App Dist.	App en canal Alfa/Beta	Despliegue sin errores de API.
8. Operate	Mantenimiento e infraestructura.	Firebase Console	Endpoints activos	Endpoint '/health' responde 200.
9. Monitor	Observabilidad y registro de fallas.	Sentry / Crashlytics	Dashboards / Alertas	Alerta configurada ante errores $> 1\%$.
10. Feedback	Captura de métricas y hallazgos.	GitHub Issues	Tareas de Refinamiento	Incidente priorizado en Backlog.

Cuadro 3: Definición de etapas del flujo DevOps para RentaFácil.

5 Estrategia de Versionamiento y Colaboración

5.1 Modelo de Ramificación (GitFlow Ligero)

- **main:** Rama de código en producción. Protegida; exige revisión de pares y paso exitoso de CI.
- **develop:** Rama de integración continua. Estado listo para la siguiente versión de prueba.

- feature/<issue-id>-<desc-corta>: Ramas temporales para desarrollo (ej. feature/US01-login-)
- hotfix/<issue-id>-<desc-corta>: Correcciones urgentes extraídas de main que se fusionan en main y develop.

5.2 Convención de Commits (Conventional Commits)

Se exige el siguiente formato estricto en los mensajes de commit:

<tipo>(<alcance>): <descripción en presente y minúsculas>

Ejemplos: feat(auth): agregar inicio de sesion, fix(search): corregir rango de precio, test(kyc): agregar prueba unitaria.

5.3 Versionamiento Semántico (SemVer)

Formato de etiquetado: vMAJOR.MINOR.PATCH (ej. v0.1.0 para el prototipo funcional del Sprint 0).

6 Integración y Entrega Continua (CI/CD)

6.1 Pipeline Automatizado de CI (GitHub Actions)

El flujo se activa ante cada push o pull_request hacia develop y main.

Workflow .github/workflows/ci.yml

```
name: Flutter CI Pipeline
on:
  push:
    branches: [ develop, main ]
  pull_request:
    branches: [ develop, main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: 'zulu'
          java-version: '17'
      - uses: subosito/flutter-action@v2
        with:
          channel: 'stable'
          cache: true
      - name: Install Dependencies
        run: cd app && flutter pub get
      - name: Linter Check
        run: cd app && flutter analyze
      - name: Run Tests with Coverage
        run: cd app && flutter test --coverage
      - name: Build Debug APK
```

```
run: cd app && flutter build apk --debug
```

6.2 Criterios de Merge

1. 100 % de los Status Checks de GitHub Actions en color verde. 2. Al menos 1 aprobación explícita de un desarrollador revisor. 3. Ausencia de conflictos de fusión con la rama de destino.

7 Estrategia de Pruebas

- **Pruebas Unitarias:** Validación de casos de uso y repositorios con cobertura $\geq 80\%$.
- **Pruebas de Widgets / UI:** Verificación de renderizado correcto de vistas críticas.
- **Criterios de Entrada y Salida:** Entrada con código compilable; Salida con 100 % de pruebas en verde y cero advertencias de linter.

8 Estrategia de Despliegue

- **Ambiente Alfa / Beta:** Despliegue automático a **Firebase App Distribution** tras cada fusión en `develop`.
- **Ambiente de Producción:** Despliegue a Google Play Store / App Store al crear un Tag `vX.Y.Z` en la rama `main`.

9 Monitoreo, Métricas y Operabilidad

Integración del SDK de **Sentry** y **Firebase Crashlytics**. SLO de Tasa de Errores: $\geq 99,2\%$ de usuarios libres de fallos. SLI de Latencia de API: 95 % de peticiones con respuesta < 300 ms.

10 Matriz de Riesgos y Mitigaciones

Riesgo Identificado	Impacto	Prob.	Plan de Mitigación
Compilación rota en rama principal (main).	Alto	Baja	Bloqueo estricto de merges sin status checks validados.
Expiración de llaves/certificados de firma.	Alto	Media	Recordatorio semestral y almacenamiento en GitHub Secrets.
Saturación de límites de cuota en APIs externas.	Medio	Media	Uso de caché local y Mocks durante pruebas integradas.

Cuadro 4: Matriz de evaluación de riesgos del pipeline.

11 Runbook Corto de Respuesta a Fallas Comunes

Incidente 1: Build de CI Roto en GitHub Actions

1. Identificar el paso fallido en la pestaña ".Actions" de GitHub.
2. Si es falla de Linter: ejecutar `flutter analyze` localmente y corregir advertencias.
3. Si es falla de Test: ejecutar `flutter test` localmente para aplicar fix en la rama `feature/`.

Incidente 2: Pico de Crashes en Producción (> 1 %)

1. Consultar dashboard de Sentry para identificar la traza del error.
2. Abrir canal de emergencia `#ops` en Slack.
3. Crear rama `hotfix/`, aplicar el parche y publicar tag de parche `vX.Y.Z+1`.

12 Investigación, Selección y Normas de Comunicación

12.1 Matriz Comparativa de Herramientas de Comunicación

Criterio de Evaluación (Puntaje 1 - 5)	Slack	MS Teams	Discord	Google Chat
Canales e Hilos de conversación	5	4	4	3
Integración con Calendario / Reuniones	4	5	3	4
Integraciones Dev (GitHub, Jira, CI)	5	4	4	3
Seguridad, Permisos y Compliance	4	5	3	4
Límite de Historial (Plan Gratuito)	3	3	5	4
Facilidad Móvil / UX	5	3	4	3
PUNTAJE TOTAL	26	24	23	21

Cuadro 5: Matriz de selección de la herramienta de comunicación del equipo.

12.2 Dictamen de Selección (≤ 300 palabras)

Se selecciona **Slack** como la herramienta oficial de comunicación para el proyecto RentaFácil debido a sus capacidades superiores de integración nativa con GitHub (notificaciones automáticas de PRs, CI/CD e issues), su estructura sólida de hilos que evita la dispersión de discusiones técnicas, y la calidad de sus aplicaciones móviles.

Mitigación de Riesgos: Para mitigar la limitación del historial de 90 días del plan gratuito, se establece la norma estricta de respaldar todas las decisiones de arquitectura, minutas de sprint y documentación técnica directamente en el repositorio de GitHub (carpeta `/docs` y `README.md`), limitando Slack únicamente a la comunicación operativa síncrona.

12.3 Configuración de Canales y Playbook de Uso

- `#anuncios`: Avisos oficiales y metas de Sprint (solo lectura).
- `#dev`: Coordinación de código Flutter y REST API.
- `#qa`: Reporte de bugs y ejecución de pruebas.

- #ops: Alertas automáticas de CI/CD (GitHub Actions) y Sentry.
- #dudas: Consultas técnicas y soporte mutuo.

Normas y SLAs de Comunicación

Horario y SLA: Atención hábil de 09:00 a 18:00 hrs. Respuesta máxima a consultas en #dev / #dudas de 2 horas. Atenciones en #ops inmediatas.

Uso de Hilos: Obligatorio responder dentro del hilo de la publicación original.

Menciones: @channel / @here exclusivo para incidentes críticos bloqueantes.